UNIVERSIDADE FEDERAL DE SERGIPE

Divisão e conquista

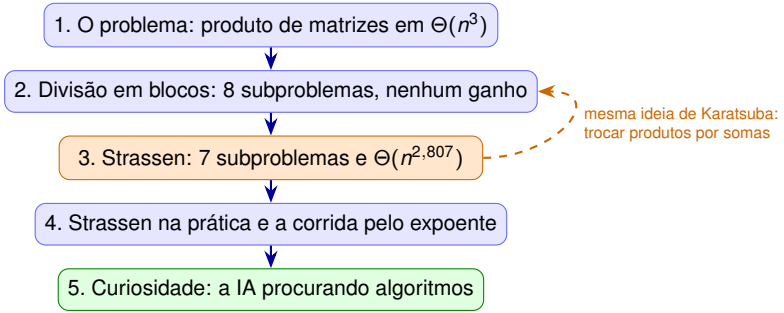
Multiplicação de matrizes de Strassen

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

4 de novembro de 2026

O que você vai aprender hoje



O problema

Por que precisamos multiplicar matrizes depressa?

Problema:

O produto de matrizes está no núcleo de muita computação:

- **grafos:** se A é a matriz de adjacência, $(A^k)_{ij}$ conta os passeios de comprimento k de i a j; $\mathrm{tr}(A^3)/6$ conta os triângulos de um grafo não dirigido;
- **redes neurais:** cada camada densa é um produto de matrizes;
- **computação gráfica, sistemas lineares, simulação.**

Para aprofundar: Cormen et al. (2009), §4.2 [3]; Strassen (1969) [16].

Solução?

O algoritmo da escola faz n^3 multiplicações. Para $n = 10^4$ são 10^{12} operações: cerca de 17 minutos a 10^9 operações por segundo.

Pergunta da aula: dá para fazer com *menos* que n^3? Até 1969, achava-se que n^3 era o mínimo possível.

Produto de matrizes: definição

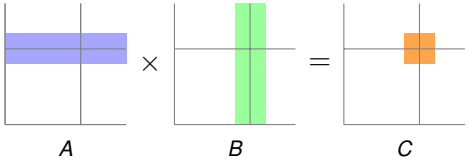
Definição

Dadas A e B de ordem $n \times n$, o produto $C = AB$ é

$$c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}, \qquad 1 \leq i, j \leq n.$$

Intuição

Cada uma das n^2 entradas de C é o produto escalar de uma *linha* de A com uma *coluna* de B: n multiplicações cada, n^3 no total.



linha i de A · coluna j de $B = c_{ij}$

Para aprofundar: Cormen et al. (2009), §4.2, procedimento SQUARE-MATRIX-MULTIPLY [3].

O algoritmo clássico em C

```c
/* C = A B, matrizes n x n em ordem de
    linhas */
void multClassico( int n, const double *A,
                   const double *B, double
    *C) {
   for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j) {
         double s = 0.0;
         for (int k = 0; k < n; ++k)
            s += A[i*n+k] * B[k*n+j];
         C[i*n+j] = s;
      }
}
```

Contagem exata:

- multiplicações: $n \cdot n \cdot n = n^3$;
- somas: $n^2(n-1)$;
- total: $2n^3 - n^2 \in \Theta(n^3)$.

Dobrar n multiplica o tempo por $2^3 = 8$.
Você vai medir isso no tutorial.

Para aprofundar: Levitin (2012), §2.3, Exemplo 3 [13]; Cormen et al. (2009), §4.2 [3].

Recapitulando: o problema

O que aprendemos

- c_{ij} é o produto da linha i de A pela coluna j de B.
- O algoritmo clássico faz n^3 multiplicações e $n^2(n-1)$ somas: $\Theta(n^3)$.
- Ler a entrada já custa $\Theta(n^2)$, então nenhum algoritmo faz menos que isso. A pergunta é onde fica o custo *entre* n^2 e n^3.

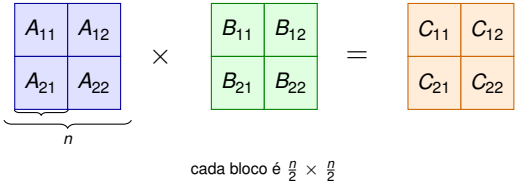
Para aprofundar: Cormen et al. (2009), §4.2 [3].

Divisão em blocos

Por que tentar divisão e conquista?

Problema:
Mergesort e o par mais próximo nos ensinaram a quebrar a entrada em pedaços menores. Uma matriz $n \times n$ (com n potência de 2) se parte naturalmente em quatro **blocos** de $\frac{n}{2} \times \frac{n}{2}$.

Solução?
Tratar cada bloco como se fosse um número e usar a regra do produto 2×2.



Para aprofundar: Cormen et al. (2009), §4.2, SQUARE-MATRIX-MULTIPLY-RECURSIVE [3].

Produto por blocos: definição

Regra do produto por blocos

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \qquad C_{12} = A_{11}B_{12} + A_{12}B_{22},$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \qquad C_{22} = A_{21}B_{12} + A_{22}B_{22}.$$

Intuição

É a mesma fórmula do produto de matrizes 2×2 de números, só que cada "número" agora é um bloco. Ela vale porque não usamos a comutatividade: escrevemos sempre $A_{\cdot\cdot}B_{\cdot\cdot}$, nunca $B_{\cdot\cdot}A_{\cdot\cdot}$.

Custo: 8 produtos de blocos (recursivos) e **4** somas de blocos, cada uma $\Theta\big((n/2)^2\big)$.

Para aprofundar: Cormen et al. (2009), §4.2 [3]; Levitin (2012), §5.4 [13].

Análise: o teorema mestre diz que nada mudou

$$T(n) = 8\,T(n/2) + \Theta(n^2)$$

$a = 8$, $b = 2$, $d = 2$: como $8 > 2^2$, as **folhas dominam**:

$$T(n) \in \Theta(n^{\log_2 8}) = \Theta(n^3).$$

Na árvore, o nível j tem 8^j nós de custo $(n/2^j)^2$:

$$8^j \left(\frac{n}{2^j}\right)^2 = n^2\,2^j,$$

uma série que *dobra* a cada nível. As n^3 folhas fazem todo o trabalho.



Lição: como as folhas dominam, só ajuda reduzir a, o *número* de subproblemas.

Para aprofundar: Cormen et al. (2009), §4.2 e §4.5, Teorema 4.1 [3]; Levitin (2012), §5.4 [13].

Recapitulando: divisão em blocos

O que aprendemos

- A regra 2×2 vale para blocos, porque não depende de comutatividade.
- 8 produtos de $\frac{n}{2}$ e somas $\Theta(n^2)$: $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.
- Quando as folhas dominam, baratear a combinação não adianta. É preciso cortar subproblemas, como Karatsuba fez com inteiros ($4 \to 3$).

Para aprofundar: Levitin (2012), §5.4 [13].

O algoritmo de Strassen

Contexto: em 1969, todos apostavam no n^3

1965 Klyuyev e Kokovkin-Shcherbak: eliminação gaussiana é ótima *se* só operarmos linhas e colunas inteiras

1968 Winograd: produto interno com metade dos produtos, mas usando $ab = ba$ (não serve para blocos)

1969 **Strassen tenta provar que 8 é o mínimo para 2×2 . . . e acha 7**

1971 Winograd; Hopcroft e Kerr: 7 é ótimo para 2×2

Como foi descoberto:

Strassen queria provar, por eliminação exaustiva de casos, que **não existia** receita 2×2 com menos de 8 produtos. A prova não fechava, e o motivo era que a receita existia.

O título é uma provocação:
Gaussian elimination is not optimal (3 páginas). Com o mesmo truque, inversão, determinante e sistemas lineares também caem para $O(n^{2,81})$, derrubando o $\Theta(n^3)$ da eliminação gaussiana.

O artigo abriu a pergunta “qual é o menor expoente ω?”. Em out./2026 a OpenAI anunciou $\omega \leq 9/4$; ninguém sabe se $\omega = 2$.

Para aprofundar: Strassen (1969) [16]; Landsberg (2008), Obs. 1.1.1 [12]; Klyuyev e Kokovkin-Shcherbak (1965) [11]; Hopcroft e Kerr (1971) [9]; OpenAI (2026) [15].

Por que 7 produtos fariam diferença?

Problema:
Um produto de blocos é caro (é ele mesmo um produto de matrizes, resolvido recursivamente); uma soma de blocos custa só $\Theta(n^2)$. Os produtos são caros e as somas são baratas.

Solução (Strassen, 1969):
calcular o produto 2×2 com **7** multiplicações, pagando com mais somas (18 em vez de 4).

	produtos	somas
blocos ingênuo	8	4
Strassen	7	18

Trocar 1 produto por 14 somas *parece* um mau negócio. Mas o produto se repete em todos os níveis da recursão, enquanto as somas são só $\Theta(n^2)$ por nível.

Para aprofundar: Strassen (1969) [16]; Levitin (2012), §5.4 [13].

Strassen: definição

Os 7 produtos

$$
\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
M_2 &= (A_{21} + A_{22})\, B_{11} \\
M_3 &= A_{11}\, (B_{12} - B_{22}) \\
M_4 &= A_{22}\, (B_{21} - B_{11}) \\
M_5 &= (A_{11} + A_{12})\, B_{22} \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}
$$

As combinações

$$
\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7 \\
C_{12} &= M_3 + M_5 \\
C_{21} &= M_2 + M_4 \\
C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

Somas: 10 nos M_i e 8 nos C_{ij}, total **18**. Nenhum produto usa comutatividade, então a receita vale para blocos.

Para aprofundar: Strassen (1969) [16]; Levitin (2012), §5.4 [13].

Por que funciona? Conferindo duas entradas

C_{12}: os termos extras se cancelam

$$M_3 + M_5 = A_{11}B_{12} \cancel{- A_{11}B_{22}} + \cancel{A_{11}B_{22}} + A_{12}B_{22} = A_{11}B_{12} + A_{12}B_{22}. \checkmark$$

C_{11}: expandindo os quatro produtos

$$\begin{aligned} M_1 &= A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} \\ M_4 &= A_{22}B_{21} - A_{22}B_{11} \\ -M_5 &= -A_{11}B_{22} - A_{12}B_{22} \\ M_7 &= A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22} \end{aligned}$$

Somando, sobra apenas $A_{11}B_{11} + A_{12}B_{21}$. ✓ C_{21} e C_{22} ficam como exercício.

Para aprofundar: Cormen et al. (2009), §4.2, verificação de C_{11} [3].

Como chegar nessas contas? A ideia

O truque de Gauss: $(a+bi)(c+di)$ com 3 produtos

Queremos $ac-bd$ e $ad+bc$, que pedem 4 produtos. Com somas antes de multiplicar:

$$k_1 = c\,(a+b), \quad k_2 = a\,(d-c), \quad k_3 = b\,(c+d)$$

$$ac - bd = k_1 - k_3, \qquad ad + bc = k_1 + k_2$$

O produto k_1 serve às **duas** saídas.

A estratégia, aplicada a 2×2:

- Um produto de somas gera vários termos $A_{ik}B_{lj}$ de uma vez.
- Cada produto deve entregar termos úteis a dois blocos de C.
- O lixo de um produto é cancelado por outro produto que *também* seja útil.

Reconstrução didática

O artigo de Strassen tem 3 páginas e não mostra a derivação. Os próximos passos mostram *um* caminho que leva às mesmas fórmulas.

Para aprofundar: Cormen et al. (2009), Exercício 4.2-7, truque de Gauss [3]; Strassen (1969) [16].

Reconstrução, passo 1: um produto, dois blocos

O alvo

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$
$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$
$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

$A_{11}B_{11}$ e $A_{22}B_{22}$ estão em blocos diferentes. Um único produto pode gerar os dois?

Diagonal vezes diagonal:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}) = A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22}$$

Dois termos úteis (para C_{11} e C_{22}) e dois de lixo.

Limpeza útil: cada lixo vira parte de um produto que também gera um termo de C.

$$M_2 = (A_{21} + A_{22})\,B_{11} = A_{21}B_{11} + A_{22}B_{11} \quad \text{útil para } C_{21}$$
$$M_5 = (A_{11} + A_{12})\,B_{22} = A_{11}B_{22} + A_{12}B_{22} \quad \text{útil para } C_{12}$$

Para aprofundar: Reconstrução das fórmulas de Strassen (1969) [16]; alvo em Cormen et al. (2009), §4.2 [3].

Reconstrução, passo 2: os blocos fora da diagonal

$C_{12} = A_{11}B_{12} + A_{12}B_{22}$

M_5 já traz $A_{12}B_{22}$, mas com $A_{11}B_{22}$ de sobra. Falta

$$A_{11}B_{12} - A_{11}B_{22} = A_{11}(B_{12} - B_{22}) = M_3$$

$$\Rightarrow \; C_{12} = M_3 + M_5$$

$C_{21} = A_{21}B_{11} + A_{22}B_{21}$

M_2 já traz $A_{21}B_{11}$, mas com $A_{22}B_{11}$ de sobra. Falta

$$A_{22}B_{21} - A_{22}B_{11} = A_{22}(B_{21} - B_{11}) = M_4$$

$$\Rightarrow \; C_{21} = M_2 + M_4$$

A correção que falta tem um fator comum (A_{11} ou A_{22}), então custa **um** produto só. **Placar:** 5 produtos ($M_1, \ldots, M_5$), dois blocos de C prontos. Para os dois blocos restantes sobram 2 produtos.

Para aprofundar: Reconstrução das fórmulas de Strassen (1969) [16].

Reconstrução, passo 3: o resíduo fatora

C_{11}**:** parta de M_1, tire o lixo $A_{11}B_{22}$ com $-M_5$ e o lixo $A_{22}B_{11}$ com $+M_4$:

$$M_1 + M_4 - M_5 = A_{11}B_{11} + A_{22}B_{22} + A_{22}B_{21} - A_{12}B_{22}$$

O que falta para chegar em C_{11}:

$$\begin{aligned} &A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22} \\ &= (A_{12} - A_{22})(B_{21} + B_{22}) = M_7 \end{aligned}$$

C_{22}**,** por simetria:

$$C_{22} - (M_1 - M_2 + M_3) = (A_{21} - A_{11})(B_{11} + B_{12}) = M_6$$

Onde está o pulo do gato

Um resíduo de 4 termos costuma exigir **2** produtos, e aí voltaríamos a 8. Aqui ele fatora como **um** produto de somas. A parte difícil é escolher o M_1 que faz os resíduos fatorarem.

E não dá para fazer melhor

Para 2×2, 7 produtos é o mínimo (Winograd; Hopcroft e Kerr, 1971). A seção final mostra a mesma busca como decomposição de um tensor.

Para aprofundar: Reconstrução das fórmulas de Strassen (1969) [16]; Winograd (1971) [18]; Hopcroft e Kerr (1971) [9].

Exemplo: Strassen em uma matriz 2×2

$$A = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}, \quad B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}$$

Pelo algoritmo clássico:

$$C = \begin{pmatrix} 1 \cdot 6 + 3 \cdot 4 & 1 \cdot 8 + 3 \cdot 2 \\ 7 \cdot 6 + 5 \cdot 4 & 7 \cdot 8 + 5 \cdot 2 \end{pmatrix} = \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}$$

com 8 multiplicações.

Por Strassen: 7 multiplicações

$$M_1 = (1+5)(6+2) = 48 \qquad M_5 = (1+3) \cdot 2 \quad = 8$$
$$M_2 = (7+5) \cdot 6 \quad = 72 \qquad M_6 = (7-1)(6+8) = 84$$
$$M_3 = 1 \cdot (8-2) \quad = 6 \qquad M_7 = (3-5)(4+2) = -12$$
$$M_4 = 5 \cdot (4-6) \quad = -10$$

$$C_{11} = 48 - 10 - 8 - 12 = 18 \qquad C_{12} = 6 + 8 = 14$$
$$C_{21} = 72 - 10 = 62 \qquad C_{22} = 48 - 72 + 6 + 84 = 66 \ \checkmark$$

Para aprofundar: Cormen et al. (2009), Exercício 4.2-1 [3].

O algoritmo recursivo

```
STRASSEN(A, B, n):      /* n potencia de 2 */
   se n <= CORTE: devolva CLASSICO(A, B, n)
   particione A, B em blocos n/2 x n/2
   M1 = STRASSEN(A11+A22, B11+B22, n/2)
   M2 = STRASSEN(A21+A22, B11,     n/2)
   ...                          /* M3 a M6 */
   M7 = STRASSEN(A12-A22, B21+B22, n/2)
   C11 = M1+M4-M5+M7;   C12 = M3+M5
   C21 = M2+M4;         C22 = M1-M2+M3+M6
   devolva C
```

Dividir: formar as 10 somas de entrada, $\Theta(n^2)$.

Conquistar: 7 chamadas recursivas de tamanho $n/2$.

Combinar: 8 somas, $\Theta(n^2)$.

A versão completa em C, com um **ponto de corte** para o algoritmo clássico, está no tutorial.

Para aprofundar: Cormen et al. (2009), §4.2, passos 1–4 [3].

Análise: o teorema mestre agora ajuda

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

$a = 7$, $b = 2$, $d = 2$: $7 > 4$, as folhas dominam:

$$T(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807}).$$

Contando só multiplicações, com $M(1) = 1$:

$$M(n) = 7\,M(n/2) \;\Rightarrow\; M(2^k) = 7^k = n^{\log_2 7}.$$

n	clássico n^3	Strassen 7^k	razão
$2^4 = 16$	$4{,}1 \times 10^3$	$2{,}4 \times 10^3$	1,7
$2^7 = 128$	$2{,}1 \times 10^6$	$8{,}2 \times 10^5$	2,5
$2^{10} = 1024$	$1{,}1 \times 10^9$	$2{,}8 \times 10^8$	3,8
$2^{13} = 8192$	$5{,}5 \times 10^{11}$	$9{,}7 \times 10^{10}$	5,7

A razão é $(8/7)^k$: cresce com n, mas devagar. O ganho só aparece em matrizes grandes.

Para aprofundar: Levitin (2012), §5.4 [13]; Cormen et al. (2009), §4.2 e §4.5 [3].

Recapitulando: Strassen

O que aprendemos

- 7 produtos e 18 somas no lugar de 8 produtos e 4 somas.
- As fórmulas não usam comutatividade, então valem recursivamente para blocos.
- $T(n) = 7T(n/2) + \Theta(n^2) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$: o *expoente* cai, e não só a constante.
- Mesmo truque de Karatsuba: aceitar mais somas (baratas) para fazer menos multiplicações recursivas (caras).

Para aprofundar: Strassen (1969) [16]; Levitin (2012), §5.4 [13].

Strassen na prática

Por que ninguém roda Strassen até $n = 1$?

Problema:

As 18 somas custam caro em matrizes pequenas. Um nível de Strassen sobre blocos de ordem m, com o clássico embaixo, vence o clássico direto quando

$$7(2m^3 - m^2) + 18m^2 \;<\; 2(2m)^3 - (2m)^2 \iff m > 7{,}5.$$

Em contagem de operações, só vale recursar a partir de $n = 16$.

Solução:

- **ponto de corte:** abaixo de um n_0, use o clássico. Na prática n_0 fica entre dezenas e centenas, por causa de cache e alocação;
- n **que não é potência de 2:** complete com zeros até a próxima potência (*padding*) ou trate a linha e a coluna que sobram à parte.

Para aprofundar: Higham (2002), cap. 23 [8]; Cormen et al. (2009), §4.2, Exercício 4.2-3 [3].

Os custos escondidos

Memória

Cada nível aloca blocos temporários para as somas e para os M_i. O clássico trabalha no lugar. Em hardware moderno, mover dados custa mais que multiplicar.

Estabilidade numérica

Em ponto flutuante, o erro de Strassen é limitado só em *norma*, não entrada a entrada. Entradas pequenas de C podem sair com erro relativo grande, o que não acontece no clássico.

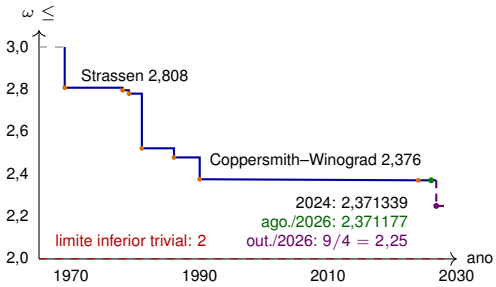
Para aprofundar: Higham (2002), cap. 23 [8].

Onde Strassen aparece de verdade

- álgebra exata (inteiros, corpos finitos): não há erro de arredondamento e o ganho assintótico vale integralmente;
- matrizes grandes e densas, com um ou dois níveis de recursão sobre um núcleo clássico otimizado.

As bibliotecas BLAS de uso geral usam, em geral, o algoritmo clássico organizado em blocos para aproveitar o cache.

A corrida pelo expoente ω



ω é o menor expoente tal que existe algoritmo $O(n^{\omega+\varepsilon})$.

De 1990 a ago./2026, décadas de trabalho baixaram ω só na *terceira* casa decimal. Em out./2026 a OpenAI anunciou $\omega \leq 9/4$: se confirmado, o maior salto desde 1981.

Esses algoritmos são **galácticos**: a constante é tão grande que só ganhariam em matrizes maiores do que qualquer computador comporta. **Ninguém sabe se $\omega = 2$.**

Para aprofundar: Strassen (1969) [16]; Alman et al. (2024), $\omega < 2{,}371339$ [1]; Dupont et al. (2026) [6]; OpenAI (2026), $\omega \leq 9/4$ [15].

Recapitulando: Strassen na prática

O que aprendemos

- Implementações reais param a recursão num ponto de corte e usam o clássico dali para baixo.
- Memória extra e estabilidade em ponto flutuante limitam o uso; em aritmética exata, Strassen brilha.
- O melhor limite anunciado para o expoente é $\omega \leq 9/4$ (out./2026, ainda em verificação), sempre com algoritmos impraticáveis. Strassen e suas variantes continuam sendo os que se usam na prática.

Para aprofundar: Higham (2002), cap. 23 [8]; OpenAI (2026) [15].

Curiosidade: a IA procurando algoritmos

Por que procurar com o computador?

Problema:
Winograd (1971) provou que **7 é ótimo** para o produto 2×2: não existe receita com 6. Para baixar o expoente é preciso um bloco base maior, 3×3, 4×4, ..., e o espaço de receitas cresce de forma explosiva.

Ideia:
aplicar Strassen duas vezes dá uma receita 4×4 com $7^2 = \mathbf{49}$ produtos. Existe alguma com menos? Ninguém sabia desde 1969.

A DeepMind transformou essa pergunta num **jogo** e deixou uma IA jogar.

Cuidado: o algoritmo precisa ser *não comutativo*. Receitas que usam $ab = ba$ (Winograd, 1968, já fazia 4×4 com 48) não servem para blocos, porque $XY \neq YX$ em matrizes.

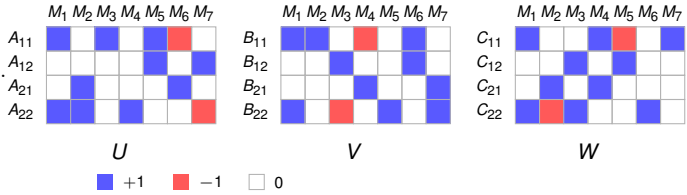
Para aprofundar: Winograd (1971) [18]; Fawzi et al. (2022) [7].

Multiplicar matrizes é decompor um tensor

Toda receita do tipo Strassen é dada por três tabelas U, V, W:

$$M_r = \Big(\sum_i u_{ir}\, a_i\Big)\Big(\sum_j v_{jr}\, b_j\Big), \quad c_k = \sum_r w_{kr}\, M_r.$$

O número de colunas R é o número de multiplicações. Achar U, V, W com R pequeno é achar uma **decomposição de posto** R do tensor $\mathcal{T}$ do produto de matrizes: $\mathcal{T} = \sum_{r=1}^{R} u_r \otimes v_r \otimes w_r$.



Strassen como decomposição de posto 7. Ex.: a coluna M_1 de U é $A_{11} + A_{22}$.

Para aprofundar: Fawzi et al. (2022), Fig. 1 [7]; Strassen (1969) [16].

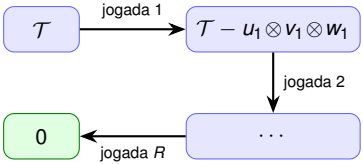
AlphaTensor (DeepMind, 2022): o TensorGame

O jogo

- **estado:** o tensor que falta decompor (começa em $\mathcal{T}$);
- **jogada:** escolher u, v, w com coeficientes em $\{-2, \ldots, 2\}$ e subtrair $u \otimes v \otimes w$;
- **vitória:** chegar ao tensor zero; cada jogada custa um ponto.

O agente aprende por reforço, como o AlphaZero no xadrez e no go. Número de jogadas = número de multiplicações.

Para aprofundar: Fawzi et al. (2022), *Nature* 610:47–53 [7].



Para 2×2, o agente redescobre $R = 7$.

O que AlphaTensor e AlphaEvolve encontraram

receita base	produtos	expoente recursivo	vale em
clássico 2×2	8	3	qualquer anel
Strassen 2×2 (1969)	7	$\log_2 7 \approx 2{,}807$	qualquer anel
Strassen 2 níveis, 4×4	49	$\log_4 49 \approx 2{,}807$	qualquer anel
AlphaTensor 4×4 (2022)	47	$\log_4 47 \approx 2{,}777$	só módulo 2 ($\mathbb{Z}_2$)
AlphaEvolve 4×4 (2025)	48	$\log_4 48 \approx 2{,}792$	complexos
Dumas, Pernet, Sedoglavic (2025)	48	$\log_4 48 \approx 2{,}792$	racionais (e reais)

- AlphaTensor também achou 5×5 módulo 2 com 96 produtos (antes 98); semanas depois, Kauers e Moosbauer chegaram a 95 com uma busca sem aprendizado.
- AlphaEvolve (um agente que evolui código com o Gemini) fez a primeira melhora sobre os 49 de Strassen para números complexos em 56 anos.
- AlphaTensor também otimizou para hardware específico: 10–20% mais rápido em GPU V100 e TPU para matrizes grandes.

Para aprofundar: Fawzi et al. (2022) [7]; Kauers e Moosbauer (2022) [10]; Novikov et al. (2025) [14]; Dumas et al. (2025) [5].

2026: AlphaEvolve também baixa o limite de ω

Rota 1: receita base pequena (Strassen, AlphaTensor)
$k \times k$ com R produtos $\Rightarrow$ expoente $\log_k R$.
Melhor por esta rota: da ordem de 2,77. *Algoritmos práticos.*

Rota 2: método laser (Strassen 1986, Coppersmith–Winograd 1990)
tensor estruturado $\rightarrow$ potências enormes $\rightarrow$ zerar blocos até sobrarem muitos produtos de matrizes independentes.
O limite de ω sai de um **problema de otimização** gigante. *Algoritmos galácticos.*

ago./2026: melhor otimizador para a Rota 2 $\Rightarrow \omega <$ 2,371177 (antes 2,371339)

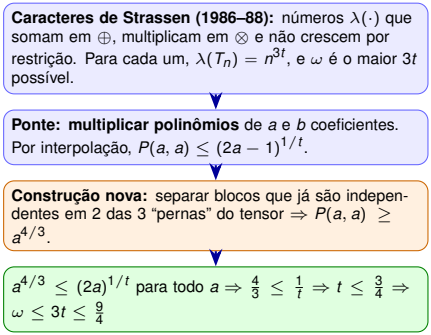
O que foi feito:

DeepMind com Alman, Vassilevska Williams e Zhou (autores dos recordes de 2023–2024). Eles reformularam a otimização da *combination loss analysis* para resolvê-la num cenário maior, criaram um otimizador com técnicas de aprendizado de máquina e o refinaram com o AlphaEvolve.

O que isso significa:
A IA não achou uma receita nova: ela melhorou o *solver* que calcula quanto o método laser consegue garantir. O ganho está na 4.ª casa decimal. É um pré-print (ago./2026), ainda sem revisão por pares.

Para aprofundar: Dupont et al. (2026), arXiv:2608.16884 [6]; Alman et al. (2024) [1]; Strassen (1988) [17]; Coppersmith e Winograd (1990) [2].

Out./2026: a OpenAI anuncia $\omega \leq 9/4$



O que mudou:

ω cairia de 2,371 para 2,25, o maior salto desde 1981. A prova não usa o método laser: volta à teoria espectral do próprio Strassen.

Quem fez:
Um modelo interno, ainda não lançado, da OpenAI. O pré-print (13 páginas, 02/10/2026) vem com uma formalização em Lean do teorema principal.

Cautela:
Ainda não houve revisão por pares nem verificação independente. O algoritmo continua galáctico: a prova não indica um tamanho de matriz em que ele seria competitivo.

Para aprofundar: OpenAI (2026), Teorema 1.1 e Seção 5 [15]; Strassen (1988) [17].

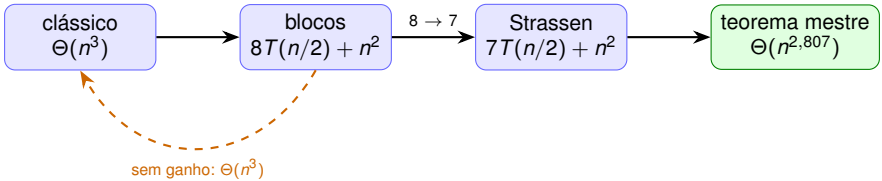
Recapitulando: a curiosidade

O que aprendemos

- Uma receita de multiplicação é uma decomposição do tensor do produto; o número de termos é o número de multiplicações.
- AlphaTensor e AlphaEvolve não inventaram uma técnica nova: eles *buscam* receitas base menores. A análise continua sendo a desta aula, por exemplo $T(n) = 48\,T(n/4) + \Theta(n^2) \in \Theta(n^{\log_4 48})$.
- Os ganhos são pequenos ($49 \to 48$) e passam longe de $\omega \approx 2{,}371$, mas são receitas *práticas*, e por meio século ninguém achou.
- Em 2026 a IA entrou na rota teórica: AlphaEvolve otimizou o método laser ($\omega < 2{,}371177$) e um modelo da OpenAI anunciou uma prova de $\omega \leq 9/4$, ainda em verificação.

Para aprofundar: Fawzi et al. (2022) [7]; Novikov et al. (2025) [14]; Dupont et al. (2026) [6]; OpenAI (2026) [15].

Fechando o ciclo



Karatsuba ($3T(n/2) + n$) e Strassen ($7T(n/2) + n^2$) seguem o mesmo roteiro: quando as folhas dominam, **cortar um subproblema muda o expoente**.

Próxima aula: programação dinâmica. E se os subproblemas se *repetem* em vez de serem independentes?

Para aprofundar: Levitin (2012), §5.4 e cap. 8 [13].

Referências I



Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou.
More asymmetry yields faster matrix multiplication, 2024.
arXiv:2404.16349; $\omega < 2{,}371339$.



Don Coppersmith and Shmuel Winograd.
Matrix multiplication via arithmetic progressions.
Journal of Symbolic Computation, 9(3):251–280, 1990.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Jean-Guillaume Dumas, Clément Pernet, and Alexandre Sedoglavic.
A non-commutative algorithm for multiplying 4×4 matrices using 48 non-complex multiplications, 2025.
arXiv:2506.13242.

Referências II



Emilien Dupont, Marvin Eisenberger, Borislav Kozlovskii, Abbas Mehrabian, Francisco J. R. Ruiz, Abigail See, Renfei Zhou, Josh Alman, Virginia Vassilevska Williams, and Matej Balog.
Improving the matrix multiplication exponent with modern optimization and AlphaEvolve, 2026.
arXiv:2608.16884; $\omega < 2{,}371177$.



Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli.
Discovering faster matrix multiplication algorithms with reinforcement learning.
Nature, 610:47–53, 2022.



Nicholas J. Higham.
Accuracy and Stability of Numerical Algorithms.
SIAM, 2 edition, 2002.
Capítulo 23: multiplicação rápida de matrizes.



John E. Hopcroft and Leslie R. Kerr.
On minimizing the number of multiplications necessary for matrix multiplication.
SIAM Journal on Applied Mathematics, 20(1):30–36, 1971.



Manuel Kauers and Jakob Moosbauer.
Flip graphs for matrix multiplication, 2022.
arXiv:2212.01175.

Referências III



V. V. Klyuyev and N. I. Kokovkin-Shcherbak.

On the minimization of the number of arithmetic operations for the solution of linear algebraic systems of equations.
USSR Computational Mathematics and Mathematical Physics, 5(1):25–43, 1965.



J. M. Landsberg.

Geometry and the complexity of matrix multiplication.
Bulletin of the American Mathematical Society, 45(2):247–284, 2008.
Observação 1.1.1: como Strassen chegou ao algoritmo.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Alexander Novikov et al.

AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025.
Google DeepMind. arXiv:2506.13131.



OpenAI.

An upper bound of $9/4$ for the matrix multiplication exponent.
OpenAI Math Release preprint, 2 out., 2026.
Produzido por um modelo interno da OpenAI; formalização em Lean no mesmo repositório; $\omega \leq 9/4$. Disponível em
`https://github.com/openai/math/blob/main/preprints/Matrix-Multiplication-Nine-Fourths-October-2-2026/paper.pdf`.

Referências IV


Volker Strassen.
Gaussian elimination is not optimal.
Numerische Mathematik, 13(4):354–356, 1969.


Volker Strassen.
The asymptotic spectrum of tensors.
Journal für die reine und angewandte Mathematik, 384:102–152, 1988.


Shmuel Winograd.
On multiplication of 2×2 matrices.
Linear Algebra and its Applications, 4(4):381–388, 1971.